Course: Advanced Databases

Group: A1

Team Members: Tobiasz Bazan, Jakub Kamiński, Adam Ćwikliński

Assignment: Partitions

1 Objective

The objective of the **Partitions** phase is to propose a logical and physical data partitioning strategy to further optimize database performance. By dividing large tables into smaller, more manageable segments, we aim to leverage **partition pruning** and **parallel execution** to reduce In/Out overhead. This phase focuses on selecting appropriate partitioning methods (Range, List, or Hash) for our specific data distribution to improve the efficiency of both analytical queries and large-scale data modifications.

2 Proposed Partitioning Strategy

2.1 Range partition (Acrtivities Table)

- **Target:** `Activities(activity_date)`
- **Logic:** The data will be partitioned by calendar quarters of the year 2026.
- **SQL Preview:** `PARTITION BY RANGE (activity_date) (PARTITION p_q1 VALUES LESS THAN (TO_DATE('01-04-2026','DD-MM-YYYY')), ...);`
- **SQL Implementation:** Creating the 'Activities_Part' table divided into 4 quarters of th year 2026.

```
CREATE TABLE Activities_Part (
    activity_id NUMBER,
    runner_id NUMBER,
    activity_date DATE,
    activity_type VARCHAR2(50),
    distance NUMBER
)
```

```
PARTITION BY RANGE (activity_date) (
    PARTITION p_2026_q1 VALUES LESS THAN (TO_DATE('01-04-2026', 'DD-MM-YYYY')),
    PARTITION p_2026_q2 VALUES LESS THAN (TO_DATE('01-07-2026', 'DD-MM-YYYY')),
    PARTITION p_2026_q3 VALUES LESS THAN (TO_DATE('01-10-2026', 'DD-MM-YYYY')),
    PARTITION p_2026_q4 VALUES LESS THAN (TO_DATE('01-01-2027', 'DD-MM-YYYY'))
);
```

2.2 List partition (Runners Table)

- **Target:** `Runners(sex)`
- **Logic:** Two distinct partitions for 'Male' and 'Female'.
- **SQL Preview:** `PARTITION BY LIST (sex) (PARTITION p_male VALUES ('M'), PARTITION p_female VALUES ('F'));`
- **SQL Implementation:** Creating the ‘Runners_Part’ table with gander partitions and a default partition of ‘p_unknown’.

```
CREATE TABLE Runners_Part (
    runner_id NUMBER,
    name VARCHAR2(100),
    sex CHAR(1)
)
PARTITION BY LIST (sex) (
    PARTITION p_male VALUES ('M'),
    PARTITION p_female VALUES ('F'),
```

```
    PARTITION p_unknown VALUES (DEFAULT)

);
```

2.3 Hash partition (Telemetry_Data Table)

- **Target:** `Telemetry_Data(activity_id)`
- **Logic:** Distribution across 4 partitions using a system hashing algorithm.
- **SQL Preview:** `PARTITION BY HASH (activity_id) PARTITIONS 4;`
- **SQL Implementation:** Creating the 'Telemetry_Part' table divided into 4 hash partitions.

```
CREATE TABLE Telemetry_Part (

    telemetry_id NUMBER,

    activity_id NUMBER,

    timestamp TIMESTAMP,

    heart_rate NUMBER

)

PARTITION BY HASH (activity_id)

PARTITIONS 4;
```

3 Purpose and Expected Improvements

- **Partition Pruning:** By using Range partitioning on dates, the DBMS will skip entire partitions that do not match the query criteria (e.g., searching for a specific month), drastically reducing disk I/O.
- **Parallel Execution:** Hash partitioning on the largest table (`Telemetry_Data`) allows the database to perform parallel scans, utilizing multiple CPU cores to process different segments of data simultaneously.
- **Improved Maintenance:** Large-scale operations like the DELETE (Q6) or UPDATE (Q5) will become more efficient as they will be localized to specific partitions.

4 Mapping to Workload

- **Q1, Q2, Q3 (Analytical SELECTs):** Will benefit from **Range Pruning**. Since these queries filter by date/year, the engine will only scan the partitions we are interested in from 2026.
- **Q5 (UPDATE):** Performance will improve by isolating the updated rows within a specific date-range partition.
- **Q6 (DELETE):** The variable performance of Q6 will be stabilized by isolating telemetry records into Hash partitions, allowing for faster row lookup and removal.

5 Planned Experiments

In the implementation phase, we plan to conduct the following experiments:

1. **Partition Pruning Efficiency:** Comparing execution times of Q1 on a non-partitioned vs. a Range-partitioned Activities table.
2. **Parallelism Test:** Measuring the throughput of Q3 using parallel execution enabled across Hash partitions of the Telemetry_Data table.

6 Summary

The proposed partitioning strategy complements our existing indexing layer by optimizing the physical storage layer. By combining Range, List, and Hash methods, we address the specific needs of our workload—from filtering by date to handling massive telemetry datasets. This dual-layer optimization (Indexes + Partitions) is expected to bring the total workload execution time well below the current baseline.